

Movie Knowledge Graph

A Linked Open Data Application

Do Tuan Hoang
20261219M

Hoang.DT261219M@sis.hust.edu.vn

Nguyen Truong Huy
20261260M

Huy.NT261260M@sis.hust.edu.vn

October 9, 2026

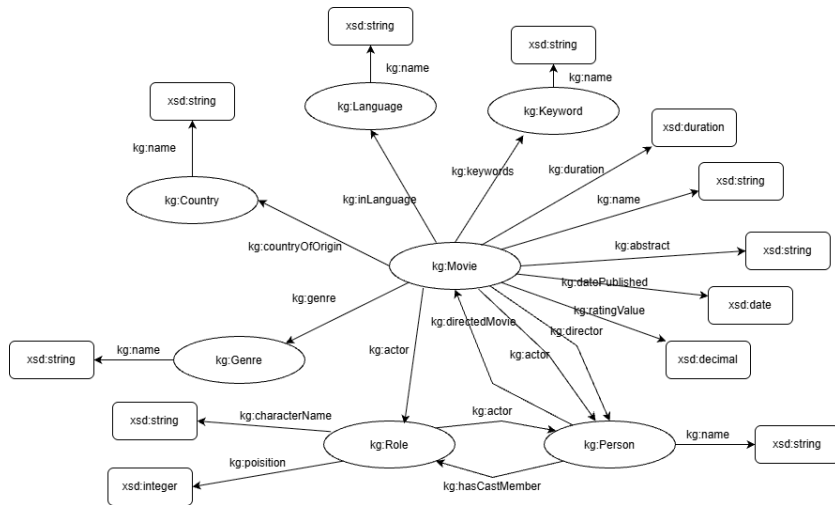
Outline

1. Define an ontology for the movie domain
2. Collect relevant data in the movie domain
3. Transform collected data into 4-star RDF
4. Establish links to other datasets
5. Provide a SPARQL query interface

Outline

1. Define an ontology for the movie domain
2. Collect relevant data in the movie domain
3. Transform collected data into 4-star RDF
4. Establish links to other datasets
5. Provide a SPARQL query interface

Define an ontology: overview



Define an ontology: classes and instances

All domain terms use `kg: = https://example.org/ontology/`.

- **Movie:** Iron Man 2, identified by TMDb ID 10138.
- **Person:** a shared actor or director resource.
- **Role:** one acting participation, with character and cast order.
- **Genre:** Drama, Action, and other genre instances.
- **Country:** countries associated with movie production.
- **Language:** original and spoken languages.
- **Keyword:** descriptive terms shared by movies.

A class is a category; an instance is a particular member of that category.

Define an ontology: properties and literals

Object properties connect resources

- Movie relationships: actor, director, genre, keywords, countryOfOrigin, inLanguage.
- Role to person: actor; movie to webpage: referencePage.
- Reasoning relationships: directedMovie, hasCastMember.

Datatype properties connect resources to literal values

- name, abstract, characterName: strings.
- datePublished: date; duration: duration.
- position: integer; ratingValue: decimal.

Define an ontology: restrictions and reasoning

- The seven classes are disjoint.
- Movie: at least one name; at most one rating value.
- Role: exactly one actor of type Person; at most one character name and position.

```
kg:directedMovie owl:inverseOf kg:director .  
kg:hasCastMember owl:propertyChainAxiom  
  (kg:directedMovie kg:actor) .
```

Person -> directedMovie -> Movie -> actor -> Person/Role

The chain derives hasCastMember. Missing-field checks are separate from OWL reasoning.

Outline

1. Define an ontology for the movie domain
2. Collect relevant data in the movie domain
3. Transform collected data into 4-star RDF
4. Establish links to other datasets
5. Provide a SPARQL query interface

Collect relevant data in the movie domain

- **Original data:** TMDB 5000 Movie Dataset on Kaggle. Movie metadata and credits in two CSV files.
- **Processed data:** *[Insert our processed-dataset link here]*. 100 movies, up to ten cast members per movie, directors only; eleven CSV tables.
- **External data:** Wikidata and DBpedia for linking local resources.

Outline

1. Define an ontology for the movie domain
2. Collect relevant data in the movie domain
3. Transform collected data into 4-star RDF
4. Establish links to other datasets
5. Provide a SPARQL query interface

Transform collected data: stable identifiers

Each entity receives a URI based on its type and source ID.

Movie ID 10138:

<https://example.org/movie/10138>

Person ID 3223:

<https://example.org/person/3223>

Genre ID 28:

<https://example.org/genre/28>

The same person URI is reused across movies and credits. Titles are display values, while source IDs identify resources.

Transform collected data: RDF triples

A movie becomes a resource with relationships and typed literals [2].

```
@prefix kg: <https://example.org/ontology/> .  
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .  
  
<https://example.org/movie/10138>  
  a kg:Movie ;  
  kg:name "Iron Man 2" ;  
  kg:datePublished "2010-04-28"^^xsd:date ;  
  kg:genre <https://example.org/genre/28> .
```

Output: 100 movies and 11,603 triples in movies.ttl.

This implements URI-based RDF. Public deployment still needs persistent, resolvable URIs in place of example.org.

Outline

1. Define an ontology for the movie domain
2. Collect relevant data in the movie domain
3. Transform collected data into 4-star RDF
4. Establish links to other datasets
5. Provide a SPARQL query interface

Establish links: matching strategy

A script finds links automatically by querying Wikidata with SPARQL.

- **Movie:** TMDB ID → title + year → manual check
- **Person:** TMDB ID → name in cast/director of a linked film → manual check
- **Country:** ISO 3166-1 code
- **Language:** ISO 639-1 code
- **Genre:** film-genre label + synonym → manual check
 - e.g. “Action” → “action film”, “Animation” → “animated film”

IDs are exact, so they come first; names are ambiguous.

Unmatched or ambiguous cases are verified by hand on Wikidata.

Establish links: represent with owl:sameAs

Links connect our resources to other datasets [1].

```
<.../movie/10138>
```

```
  owl:sameAs <http://www.wikidata.org/entity/Q205028> ,  
              <http://dbpedia.org/resource/Iron_Man_2> .
```

```
<.../genre/28>
```

```
  skos:closeMatch <http://www.wikidata.org/entity/Q188473> .
```

- Genres use the weaker `skos:closeMatch`.
- 99 movies, 805 people, and 56 lookup resources approved.
- **1,901** link triples in total.

External links provide the cross-dataset connection for the fifth star.

Outline

1. Define an ontology for the movie domain
2. Collect relevant data in the movie domain
3. Transform collected data into 4-star RDF
4. Establish links to other datasets
5. Provide a SPARQL query interface

Interface via SPARQL endpoint/terminal to query data

Our terminal interface loads movie data, the ontology, and optional link files. SPARQL selects matching graph patterns [3].

```
python scripts/ask.py -q queries/4.sparql
```

```
python scripts/ask.py -q queries/11.sparql --reasoning
```

```
python scripts/ask.py -q queries/show_links.sparql  
--links output/movie_links.ttl
```

The last example is one command, wrapped here for display. `-reasoning` adds inferred triples before the query runs.

Interface via SPARQL endpoint/terminal to query data

Do we have any Spider-Man movies?

```
PREFIX kg: <https://example.org/ontology/>
```

```
SELECT ?movie ?title WHERE {  
  ?movie a kg:Movie ; kg:name ?title .  
  FILTER(CONTAINS(LCASE(STR(?title)), "spider"))  
}  
ORDER BY ?title
```

No reasoning needed.

Interface via SPARQL endpoint/terminal to query data

Who acted in Avatar, and which characters did they play?

```
PREFIX kg: <https://example.org/ontology/>
SELECT ?person ?name ?character ?position WHERE {
  VALUES ?movie { <https://example.org/movie/19995> }
  ?movie kg:actor ?role .
  ?role a kg:Role ; kg:actor ?person ; kg:position ?position .
  ?person a kg:Person ; kg:name ?name .
  OPTIONAL { ?role kg:characterName ?character }
}
ORDER BY ?position ?name
LIMIT 10
```

Follows Movie -> Role -> Person.

Interface via SPARQL endpoint/terminal to query data

Which Drama films have the highest average rating?

```
PREFIX kg: <https://example.org/ontology/>
SELECT ?movie ?title ?average WHERE {
  ?movie a kg:Movie ;
         kg:name ?title ;
         kg:genre ?genre ;
         kg:ratingValue ?average .
  ?genre kg:name "Drama" .
}
ORDER BY DESC(?average) ?title
LIMIT 20
```

Ratings are decimal values directly on movies.

Interface via SPARQL endpoint/terminal to query data

Which Action films since 2010 feature Scarlett Johansson?

```
PREFIX kg: <https://example.org/ontology/>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
SELECT DISTINCT ?movie ?title ?date WHERE {
  VALUES ?actor { <https://example.org/person/1245> }
  VALUES ?genreName { "Action" }
  ?movie a kg:Movie ; kg:name ?title ; kg:datePublished ?date ;
    kg:actor ?actor ; kg:genre ?genre .
  ?actor a kg:Person .
  ?genre kg:name ?genreName .
  FILTER (?date >= "2010-01-01"^^xsd:date)
}
ORDER BY ?date ?title
```

Combines actor, genre, and release-date conditions.

Interface via SPARQL endpoint/terminal to query data

Which directors have at least two movies in our sample?

```
PREFIX kg: <https://example.org/ontology/>
SELECT ?director ?name (COUNT(DISTINCT ?movie) AS ?films) WHERE {
    ?movie a kg:Movie ; kg:director ?director .
    ?director a kg:Person ; kg:name ?name .
}
GROUP BY ?director ?name
HAVING (COUNT(DISTINCT ?movie) >= 2)
ORDER BY DESC(?films) ?name
```

Uses grouping and counting.

Interface via SPARQL endpoint/terminal to query data

Which movies did each person direct?

```
PREFIX kg: <https://example.org/ontology/>
SELECT ?director ?name ?movie ?title WHERE {
  ?director kg:directedMovie ?movie ; kg:name ?name .
  ?movie kg:name ?title .
}
ORDER BY ?name ?title
LIMIT 20
```

Run with `-reasoning`: uses the inverse of `kg:director`.

Interface via SPARQL endpoint/terminal to query data

Which actors appeared in movies directed by each person?

```
PREFIX kg: <https://example.org/ontology/>
SELECT DISTINCT ?director ?directorName ?actor ?actorName WHERE {
  ?director kg:hasCastMember ?actor ; kg:name ?directorName .
  ?actor a kg:Person ; kg:name ?actorName .
}
ORDER BY ?directorName ?actorName
```

Run with `-reasoning`: uses the property chain.

Interface via SPARQL endpoint/terminal to query data

Which external resources are linked to our movies?

```
PREFIX kg: <https://example.org/ontology/>
PREFIX owl: <http://www.w3.org/2002/07/owl#>

SELECT ?title ?external WHERE {
    ?movie a kg:Movie ;
           kg:name ?title ;
           owl:sameAs ?external .
}
ORDER BY ?title
```

Load `-links output/movie_links.ttl`; reasoning is not required.

Interface via SPARQL endpoint/terminal to query data

Which keywords describe Iron Man 2?

```
PREFIX kg: <https://example.org/ontology/>
SELECT ?keywordName WHERE {
  <https://example.org/movie/10138>
    kg:keywords ?keyword .
  ?keyword kg:name ?keywordName .
}
ORDER BY ?keywordName
```

Demonstrates our keyword extension; no reasoning needed.

References I

- [1] Tim Berners-Lee. Linked data. <https://www.w3.org/DesignIssues/LinkedData.html>, 2006.
- [2] W3C RDF Working Group. Rdf 1.1 concepts and abstract syntax.
<https://www.w3.org/TR/rdf11-concepts/>, 2014. W3C Recommendation.
- [3] W3C SPARQL Working Group. Sparql 1.1 query language.
<https://www.w3.org/TR/sparql11-query/>, 2013. W3C Recommendation.